ANDROMEDA RESEARCH STATION

Research Notebook

A Beginner's Guide to Quantum Computing

Restricted research notes · Diagnostic Protocol: The Last Qubit

PURPOSE

Learn just enough quantum computing to investigate a possible **bit-flip error**, read the diagnostic results, and restore the simulated computation. No prior physics or advanced maths is needed.

How to use this notebook. Entries tagged **CORE** contain everything you need to solve the puzzle. Entries tagged **GO DEEPER** explain why the method works. Read them if you are curious, or skip them and return later. Boxes marked "In this investigation" connect each idea to the console in front of you.

01 The Basics

1. Classical bit CORE

A **bit** is the basic unit of classical digital information. At any moment it holds exactly one of two values: `0` or `1`.

Value	Think of it as	Meaning
`0`	Switch OFF	One of the two possible values
`1`	Switch ON	The other possible value

Everything on an ordinary computer, such as text, photos and games, is stored as long strings of bits. A classical bit always has one definite value, and you can read it without changing it.

2. Qubit CORE

A **qubit** (quantum bit) is the basic unit of quantum information. When you read it out, you get `0` or `1`, just like a bit. The difference is what it can be before you read it: its state can be a **superposition** of the two basis states, written $|0\rangle$ and $|1\rangle$ (see entry 4).

Real qubits are physical systems, for example a tiny superconducting circuit, a trapped ion or a single photon. They follow the rules of quantum mechanics, which is why they behave differently from classical bits.

	Classical bit	Qubit
Values you can read out	`0` or `1`	`0` or `1`
State before reading	Definitely 0 or definitely 1	Can be a superposition of $\|0\rangle$ and $\|1\rangle$
Effect of reading	None	Gives a classical result and updates the qubit's state

	Classical bit	Qubit
Can an unknown state be copied?	Yes, freely	No (the no-cloning theorem)

> **COMMON MISCONCEPTION**
> A qubit is **not** a switch that is secretly 0 or 1 while we simply do not know which. Its state follows quantum rules, and measurement probabilities can differ from anything a classical bit could produce.

3. Basis states: |0⟩ and |1⟩ CORE

These are the two standard reference states of a qubit. The symbol |⟩ is called a **ket**. It is the notation physicists use to say "this is a quantum state", not just a plain number.

- |0⟩: measuring in the computational basis returns `0` with certainty.
- |1⟩: measuring in the computational basis returns `1` with certainty.

When a qubit is in |0⟩ or |1⟩, it behaves much like a classical bit. That is why this puzzle can be solved by reading bit patterns.

4. Superposition GO DEEPER

A general single-qubit state mixes the two basis states:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

Here α and β are numbers called **amplitudes** (they can be complex), and together they satisfy:

$$|\alpha|^2 + |\beta|^2 = 1$$

When the qubit is measured in the computational basis:

- the probability of getting `0` is $|\alpha|^2$;
- the probability of getting `1` is $|\beta|^2$.

Worked example. If $\alpha = \beta = 1/\sqrt{2}$, then $|\alpha|^2 = |\beta|^2 = 1/2$. Each measurement gives `0` or `1` with a 50% chance, like a fair coin. Run the same circuit many times (each run is called a **shot** in Qiskit) and about half the results will be `0`.

> **COMMON MISCONCEPTION**
> Superposition does **not** mean the qubit shows both `0` and `1` at once. A single measurement gives exactly one outcome. It also does not mean "unknown value": a superposition is a perfectly definite quantum state whose randomness appears only when you measure.

5. Measurement CORE

Measurement produces a classical result (`0` or `1`) from a quantum system. For a qubit in superposition, the result is random, with probabilities set by the state. Afterwards the qubit is left in the state that matches the result you obtained, so a measurement generally changes the qubit.

IN THIS INVESTIGATION

The diagnostic console shows results from measuring a **simulated** quantum circuit. Two kinds of measurement matter here: the two **syndrome** bits (which help you find the error) and the final **data** bits (which verify the repair).

02 Operations on Qubits

6. Quantum gate CORE

A **quantum gate** is an operation that changes the state of one or more qubits. Gates are the building blocks of every quantum circuit. Think of a gate as a single instruction, such as “flip this qubit” or “flip that qubit if another one is `1`”.

Gates are **reversible**: every gate can be undone by another operation. This matters for error correction, because a flip can be undone by flipping again.

7. The X gate (bit-flip gate) CORE

The **Pauli-X gate** swaps the two basis states:

$$X|0\rangle = |1\rangle \qquad X|1\rangle = |0\rangle$$

Input	Gate	Output
$\|0\rangle$	X	$\|1\rangle$
$\|1\rangle$	X	$\|0\rangle$

X works like a classical NOT. It also acts on superpositions by swapping the two amplitudes: $X(\alpha|0\rangle + \beta|1\rangle) = \alpha|1\rangle + \beta|0\rangle$. Applying X twice returns the qubit to where it started.

IN THIS INVESTIGATION

In this game, an X gate plays two roles: an **unintended** X models a bit-flip error, and a **deliberate** X is the correction. The gate is the same. What differs is whether it was intended.

8. The CNOT gate GO DEEPER

The **controlled-NOT (CNOT)** gate acts on two qubits, a **control** and a **target**. If the control is `1`, it flips the target. If the control is `0`, it does nothing. The control is never changed.

Control (before)	Target (before)	Control (after)	Target (after)
`0`	`0`	`0`	`0`
`0`	`1`	`0`	`1`
`1`	`0`	`1`	`1`
`1`	`1`	`1`	`0`

CNOT is how one qubit can "inspect" another. If the target starts at `0`, a CNOT adds the control's value to it (mod 2). The diagnostic circuit uses exactly this trick to compute parity checks.

9. Quantum circuit CORE

A **quantum circuit** is an ordered list of gates and measurements applied to qubits. In a circuit diagram, each qubit is a horizontal line and time runs from left to right. The table below is the station's diagnostic circuit, drawn as a grid.

Qubit	Start	Error?	CNOT 1	CNOT 2	CNOT 3	CNOT 4	Measure
q0	`0`	X?	●				
q1	`0`	X?		●	●		
q2	`0`	X?				●	
A	`0`		⊕	⊕			M
B	`0`				⊕	⊕	M

Legend: ● = control, ⊕ = target of a CNOT, X? = a possible bit-flip error, M = measurement. Ancilla A collects the parity of q0 and q1. Ancilla B collects the parity of q1 and q2.

> **IN THIS INVESTIGATION**
> Three **data** wires carry the information being protected. Two extra **helper** wires (A and B) detect disagreements between the data qubits.

03 Understanding the Fault

10. Bit-flip error CORE

A **bit-flip error** happens when a qubit's computational-basis value is swapped: $|0\rangle$ becomes $|1\rangle$, or $|1\rangle$ becomes $|0\rangle$. In the circuit model, it is an unintended X gate. In real hardware, causes include stray electromagnetic noise, heat and imperfect control pulses.

Example. The system expects a data qubit to stay in $|0\rangle$, but an error changes it to $|1\rangle$. The qubit now differs from the state the computation expects.

> **RESEARCH NOTE**
> This exercise considers a **single** bit-flip error. Quantum systems can suffer other kinds of errors too (see entry 20).

11. Why not just look at the qubits? GO DEEPER

Measuring a data qubit directly would force it to give a definite `0` or `1` and would **destroy any superposition** it carried. If the qubits hold protected quantum information, that is exactly what we cannot afford.

The solution is to ask only about **relationships**: "do q0 and q1 agree?" rather than "what is q0?" The answer to a relationship question does not depend on the hidden amplitudes, so the protected information survives.

INTUITION

Compare a proofreader who checks that two copies of a document match **without reading what it says**. They can spot a mismatch without learning, or disturbing, the content.

12. Error correction CORE

Error correction means detecting and fixing certain errors while preserving the information being protected. It follows four stages:

- **Encode:** spread one piece of information across several qubits.
- **Detect:** measure carefully chosen checks to get the syndrome.
- **Correct:** apply the operation that undoes the diagnosed error.
- **Verify:** confirm that the result is what it should be.

YOUR OBJECTIVE

Identify the error from the diagnostic results, apply the appropriate correction, and verify the outcome.

04 The Three-Qubit Repetition Code

13. Repetition code CORE

The **three-qubit repetition code** stores one **logical** bit of information redundantly across three **physical** qubits:

$$|0_\mathrm{L}\rangle = |000\rangle \qquad |1_\mathrm{L}\rangle = |111\rangle$$

The subscript L means “logical”. The logical information lives in all three qubits together, not in any single one.

INTUITION

It works like sending a message three times and taking a majority vote. If all three copies should agree, one flipped value stands out. By checking which pairs disagree, you can tell which qubit was flipped.

GO DEEPER

The code can also protect a superposition. Using CNOT gates, a qubit in the state $\alpha|0\rangle + \beta|1\rangle$ is encoded as $\boldsymbol{\alpha|000\rangle + \beta|111\rangle}$.

Encoding is not copying. This is not three copies of the original qubit (the no-cloning theorem forbids copying an unknown quantum state). It is an entangled state in which the three qubits share the information. In this game the input is the simple state $|000\rangle$, so you can reason with plain bit patterns.

14. Data qubits CORE

The **data qubits** carry the encoded information. In this game they are labelled:

- `q0`: first data qubit
- `q1`: middle data qubit

- `q2`: third data qubit

The diagnostic circuit checks relationships between neighbouring data qubits.

15. Ancilla qubits CORE

An **ancilla** is an extra helper qubit, initially set to $|0\rangle$, used to record the result of a check so we can measure the helper instead of the data. This circuit uses two:

- **Ancilla A** records the check on q0 and q1 (lamp A on the console).
- **Ancilla B** records the check on q1 and q2 (lamp B on the console).

Ancillas are **not** extra copies of the logical information.

16. Parity check CORE

A **parity check** tells you whether two bits agree or differ. It uses the XOR operation, written $\oplus$:

First bit	Second bit	XOR result	Meaning
`0`	`0`	`0`	agree
`0`	`1`	`1`	differ
`1`	`0`	`1`	differ
`1`	`1`	`0`	agree

How the circuit computes it. Ancilla A starts at `0`. One CNOT from q0 and one CNOT from q1 each flip A if their control is `1`. If q0 and q1 are both `1`, A is flipped twice and ends at `0` (they agree). If only one is `1`, A is flipped once and ends at `1` (they differ). So A ends up holding q0 $\oplus$ q1, and B ends up holding q1 $\oplus$ q2.

> GO DEEPER
> For the protected state $\alpha|000\rangle + \beta|111\rangle$, both branches have the same parities (all bits agree), so the checks return `00` **whatever α and β are** and leave the superposition intact. After an X error on q1, the state becomes $\alpha|010\rangle + \beta|101\rangle$. Both branches now give the same parities again (`11`), so the syndrome is certain and still reveals nothing about α or β.

17. Syndrome CORE

The **syndrome** is the collection of parity-check results used to diagnose an error. The word is borrowed from medicine: like symptoms, it points to the problem without showing the cause directly. In this game it is written as two bits, in this order:

$$s = (q0 \oplus q1,\ q1 \oplus q2)$$

If there is at most one bit-flip error, the two bits identify the affected data qubit. The syndrome tells you **where** the error is, not what the logical value is.

05 Reading the Diagnostic Console

Use the syndrome bits in the order shown above (first bit = lamp A, second bit = lamp B).

Syndrome	What it tells you	Action
`00`	Both pairs agree: no disagreement detected	No correction
`10`	First pair (q0, q1) differs; second pair (q1, q2) agrees	Apply X to `q0`
`11`	Both pairs differ	Apply X to `q1`
`01`	First pair agrees; second pair (q1, q2) differs	Apply X to `q2`

Why does the syndrome identify the error? Start from the encoded state |000⟩ and flip one qubit:

Flipped qubit	Data after flip	q0 vs q1	q1 vs q2	Syndrome	Fix
None	`000`	agree	agree	`00`	None
q0	`100`	differ	agree	`10`	X on q0
q1	`010`	differ	differ	`11`	X on q1
q2	`001`	agree	differ	`01`	X on q2

INTUITION

The lamp rule. Find the qubit that sits in **every** pair that differs and in **no** pair that agrees. For syndrome `10`, only the first pair differs, and q0 is the only qubit that belongs to the first pair alone. For `11`, both pairs differ, and q1 is the only qubit in both pairs. For `01`, only the second pair differs, which singles out q2.

IN THIS INVESTIGATION

The console's syndrome comes from the circuit's **measured** parity checks. It is produced by the simulation, not assigned by the story.

06 Correction and Verification

18. Correction operation CORE

Once the syndrome points to a flipped qubit, apply an **X gate** to that data qubit. For example, if q1 has flipped from `0` to `1`, applying X to q1 flips it back to `0`.

LIMITATION

If the syndrome is `00`, apply **no** correction. An X on a healthy qubit would **create** an error.

19. Verification CORE

Verification is a follow-up check that the corrected circuit meets the expected condition. Here, the encoded state started as |000⟩, so a successful repair means that measuring the three data qubits gives `000`. The console reports success only if the simulation's actual result passes this rule.

A wrong correction should fail verification, letting you re-read the syndrome and try again. Examples:

Error on	Syndrome	Your fix	Data after	Result
q1	`11`	X on q1	`000`	Verified
q1	`11`	X on q0	`110`	Fails: now two bits are wrong
None	`00`	X on q2	`001`	Fails: you created an error
q2	`01`	No correction	`001`	Fails: the error remains

> **RESEARCH NOTE**
> Checking for `000` works here because the starting state is known. In a real experiment the encoded state is usually unknown, so verification uses other methods, such as repeating the syndrome measurement.

20. Limitations of this code CORE

The three-qubit repetition code corrects a **single** bit-flip (X) error under the assumptions of this exercise. It has important limits:

- **Phase-flip errors go unnoticed.** A phase flip changes the sign, turning $\alpha|000\rangle + \beta|111\rangle$ into $\alpha|000\rangle - \beta|111\rangle$. The parity checks compare bit values only, so they cannot see it.
- **Two errors can mislead the syndrome.** If q0 and q1 both flip, the data reads `110`, the syndrome is `01`, and the "fix" (X on q2) produces `111`. The encoded value has silently changed from $|0_L\rangle$ to $|1_L\rangle$.
- **It is a teaching model.** Real quantum computers use larger codes (such as Shor's nine-qubit code or the surface code) that handle both bit-flip and phase-flip errors, repeat the checks continually, and allow for faulty measurements.

> **LIMITATION**
> A successful verification shows how **this** simulated circuit behaves. It does not prove that every kind of quantum error has been eliminated.

07 Your Mission Procedure

#	Step	What you do
1	Receive the report	A research partner reports an inconsistent result.
2	Review the notebook	Recall the X gate, bit-flip errors and parity checks.
3	Run diagnosis	Execute the diagnostic circuit using Qiskit Aer.
4	Read the syndrome	Work out which data qubit the two measured parity bits indicate.
5	Apply correction	Select the matching X operation, or no correction if the syndrome is `00`.
6	Verify the result	Check the outcome returned by the simulation. Retry if it fails.
7	Transmit the report	After verification succeeds, report the diagnosis and the correction.

FIELD NOTES: BEFORE YOU PROCEED

The X gate swaps |0⟩ and |1⟩.

The syndrome describes disagreement between neighbouring data qubits.

For a single error, the syndrome tells you which correction to try.

Only the simulation's measured result decides whether verification succeeds.

08 Check Your Understanding

Question	Answer
What does the X gate do to |0⟩?	It turns it into |1⟩.
A qubit in a 50/50 superposition is measured once. What do you see?	One result, `0` or `1`, each with 50% probability. Never both.
The console shows syndrome `01`. Which qubit flipped, and what is the fix?	q2 flipped. Apply X to q2.
The data reads `010`. What is the syndrome?	(0 ⊕ 1, 1 ⊕ 0) = `11`, which points to q1.
Why not measure the data qubits directly?	It would collapse any superposition. Parity checks reveal only relationships between qubits.
Why can this code not fix a phase flip?	The parity checks compare bit values only, and a phase flip does not change them.

09 Glossary

Term	Meaning
Amplitude	A number (α or β) that determines measurement probabilities of a basis state.
Ancilla	A helper qubit used to record a check, so data qubits need not be measured.
Bit-flip error	An error that swaps |0⟩ and |1⟩, modelled as an unintended X gate.
CNOT	Two-qubit gate that flips the target only if the control is `1`.
Data qubit	A qubit that carries the encoded information (q0, q1, q2).
Ket |⟩	Notation for a quantum state, such as |0⟩.
Logical bit	Information encoded across several physical qubits, for example |0_L⟩ = |000⟩.
Measurement	Obtaining a classical result from a quantum system.
Parity / XOR (⊕)	Tells whether two bits differ (`1`) or agree (`0`).
Qiskit / Qiskit Aer	A Python toolkit for building quantum circuits, and its local simulator.

Term	Meaning
Shot	One run of a circuit ending in a measurement.
Superposition	A quantum state combining basis states, such as $\alpha\|0\rangle + \beta\|1\rangle$.
Syndrome	The parity-check results used to locate an error.
Verification	A check that the corrected circuit gives the expected result.
X gate	The gate that swaps $\|0\rangle$ and $\|1\rangle$.

A Designer's Notes (not shown in game)

Changes made while turning the original text into this notebook:

- **Clarified syndrome wording.** "Only q0 and q1 disagree" could be misread, so it now reads "first pair differs, second pair agrees".
- **Added the CNOT entry**, since the parity circuit cannot be understood without it, and a circuit grid showing the full diagnostic.
- **Added the key quantum idea:** parity checks reveal where an error is without revealing the logical value, and encoding is not copying (no-cloning).
- **Added worked examples** of wrong corrections and of a two-error failure, to make the limitations concrete.
- **Named ancillas A and B**, matching the lamp-based puzzle in the prototype demo.
- **Tagged entries CORE or GO DEEPER** so beginners can finish the puzzle without reading every entry.
- **Developer reminder:** Qiskit prints classical bits right-to-left. Reverse the register strings before showing the syndrome, so `10` means lamp A on and lamp B off.
- **Verification caveat:** checking for `000` works only because the input state is known.